

SENIOR DevOps INTERVIEW HANDBOOK



**REAL
SCENARIOS**



**PRACTICAL
ANSWERS**



**PRODUCTION
FOCUS**



**SENIOR-LEVEL
INSIGHTS**



**MULTI-TOOL
COVERAGE**

HOW SENIOR DevOps INTERVIEWS ACTUALLY TEST YOU

IT'S NOT ABOUT TOOLS

NOT THIS

- Listing tools and features
- Memorizing commands
- Giving one-line answers
- Choosing a tool without reason
- Focus only on implementation

IT IS THIS

- Solving real production problems
- Explaining your reasoning
- Considering trade-offs
- Thinking about failure and scale
- Owning end-to-end outcomes

WHAT INTERVIEWERS WANT



Production experience and judgment



Clear and structured thinking



Ability to design, troubleshoot and improve



Understanding of trade-offs and risks



Communication and decision making



Experience with incidents and real failures

HOW TO STRUCTURE A SENIOR-LEVEL ANSWER

1

Understand the problem

Ask clarifying questions. Identify the real goal.

2

State assumptions

Call out what you are assuming is true.

3

Define requirements

List functional and non-functional requirements.

4

Propose an approach

Explain options and trade-offs. Justify your choice.

5

Consider operational impact

Security, reliability, cost, scalability and maintenance.

6

Validate and iterate

How you would test, monitor and improve.

KEY AREAS INTERVIEWERS EXPECT YOU TO COVER



Requirements – understand business, technical and operational needs



Assumptions and Constraints – call them out early



Failure Domains – what can fail and blast radius



Security – access, data protection, least privilege



Reliability – availability, redundancy, recovery



Cost – cost implications and optimization



Operability – monitoring, alerting, runbooks, team ownership



Trade-Offs – explain pros and cons, avoid “one best solution”

SENIOR SCENARIO



You inherit a production platform you did not design. What do you assess before changing anything?

EXAMPLE SENIOR ANSWER (STRUCTURE)

- 1 Understand the current architecture and business goals
- 2 Review recent changes, incidents and open risks
- 3 Assess reliability, performance and security posture
- 4 Identify critical dependencies and failure domains
- 5 Review monitoring, alerting and operational processes
- 6 Look for technical debt and quick wins
- 7 Propose changes in a low-risk, incremental way

COMMON MISTAKES AT SENIOR LEVEL

- ✗ Jumping to a solution without understanding the problem
- ✗ Ignoring non-functional requirements
- ✗ Not discussing failure scenarios
- ✗ Saying a tool is “better” without context
- ✗ Missing security or cost considerations
- ✗ Not explaining trade-offs
- ✗ Focusing only on implementation details

SENIOR TIP



- Think like an owner, not a task executor.
- Show how you make safe, incremental changes.
- Always consider failure, scale, security and cost.
- Explain trade-offs instead of claiming a single best solution.
- Use real production examples from your experience.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

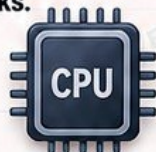
LINUX PRODUCTION TROUBLESHOOTING

GO BEYOND COMMANDS: DIAGNOSE, FIND ROOT CAUSE, FIX AND PREVENT

1 CPU SATURATION vs HIGH LOAD AVERAGE

- High CPU: CPU is busy (close to 100%).
- High load average: many processes waiting (CPU, I/O or uninterruptible sleep).
- Load can be high even when CPU is low due to I/O wait, disk, network or locks.

```
uptime
top
mpstat -P ALL 1
pidstat -u 1
```



2 MEMORY PRESSURE, SWAP AND OOM

- Memory pressure causes slow performance even before OOM.
- Check memory usage, cache, swap and OOM kills.
- Look for processes consuming memory.

```
free -h
vmstat 1
swapon --show
dmesg | grep -i oom
top -o %MEM
```



3 DISK CAPACITY vs I/O SATURATION

- Low disk space causes failures.
- High I/O wait can make the system slow even with free space.
- Check both capacity and I/O performance.

```
df -h
du -sh /var/*
iostat -xz 1
iotop -o
```



4 FILE DESCRIPTOR EXHAUSTION

- Each process has a limit on open files.
- When exhausted, services cannot open new connections or files.
- Common in high-traffic applications.

```
ulimit -n
lsof | wc -l
lsof -p <PID> | wc -l
cat /proc/sys/fs/file-max
cat /proc/<PID>/limits
```



5 PROCESSES, THREADS, SIGNALS AND SYSTEMD

- Identify what is running, stuck or restarting.
- Use signals for controlled troubleshooting.
- Use systemd to check service health and logs.

```
ps auxf
pstree -p
kill -l
systemctl status <service>
journalctl -u <service> -n 100
```



6 /PROC, JOURNALCTL, SS, LSOF, VMSTAT, IOSTAT

- /proc shows real-time kernel and process info.
- journalctl for system and service logs.
- ss for sockets, lsof for open files.
- vmstat and iostat for system performance.

```
cat /proc/loadavg
cat /proc/meminfo
cat /proc/<PID>/status
ss -tulnp
journalctl -n 100
vmstat 1
iostat -xz 1
```



7 SENIOR SCENARIO



Latency increased sharply, CPU is only 35%, and memory looks normal. Where do you investigate next?

INVESTIGATION APPROACH

- 1 Check load average and I/O wait (uptime, vmstat, iostat).
- 2 Look at disk latency and throughput (iostat, iotop).
- 3 Check network and connections (ss, netstat, timeouts).
- 4 Review application logs and recent changes (journalctl, app logs).
- 5 Check for resource limits (file descriptors, threads, connection pools).
- 6 Correlate with metrics, traces and system events.

POSSIBLE CAUSES

- High disk I/O wait
- Network latency or packet loss
- Database or external dependency slowness
- Connection pool exhaustion
- File descriptor exhaustion
- Lock contention
- Application thread starvation
- Kernel resource limits

WHAT TO CHECK

- I/O wait and disk latency
- Network and DNS
- Open connections and limits
- Application and system logs
- Recent deployments/changes
- Database performance
- External service health
- System resource limits

SENIOR TIPS



- Don't focus only on CPU and memory.
- Always check I/O, network, dependencies and limits.
- Use metrics, logs and traces to confirm the real bottleneck.
- Think about the blast radius before making changes.
- Fix the immediate issue, then find the root cause and prevent recurrence.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

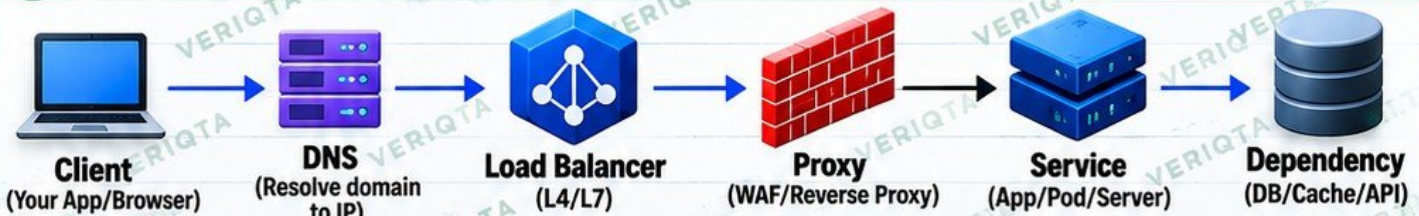


@veriqta

NETWORKING AND DNS FAILURE INVESTIGATION

FOLLOW THE REQUEST: FIND WHERE IT FAILS

1 REQUEST FLOW (HIGH LEVEL)



2 DNS RESOLUTION

- Converts domain name to IP address.
- Issues: wrong record, expired TTL, DNS server failure, split-horizon DNS.

```
dig example.com
dig example.com +short
nslookup example.com
```



3 TCP CONNECTION ESTABLISHMENT

- 3-way handshake: SYN → SYN-ACK → ACK.
- Failures: blocked port, firewall, security group, network ACL, routing issue.

```
nc -zv example.com 443
telnet example.com 443
ss -tanp | grep :443
```



4 TLS HANDSHAKE

- Validates certificate and establishes encryption.
- Issues: expired certificate, wrong domain, TLS version/cipher mismatch.

```
openssl s_client -connect example.com:443
-servername example.com
curl -lv https://example.com
```



5 ROUTING AND NAT

- Traffic is routed through subnets, NAT gateways, routers and transit networks.
- Issues: wrong route, asymmetric routing, NAT port exhaustion, firewall rules.

```
ip route
traceroute example.com
mtr example.com
```



6 CONNECTION POOLS, TIMEOUTS AND RETRIES

- Applications reuse connections for performance.
- Issues: connection pool exhaustion, short timeouts, aggressive retries can make problems worse.

```
# Example (curl with timeout)
curl --connect-timeout 5 --max-time 10
https://example.com
```



8 INTERMITTENT CONNECTIVITY

- May be caused by network devices, DNS caching, load balancer health checks or dependency issues. Always compare a working path with a failing path.

```
mtr example.com
dig example.com
curl -lv --resolve example.com:IP:443 https://example.com
```



7 MTU AND PACKET LOSS

- Large packets can be dropped if MTU is wrong.
- Issues: packet fragmentation, MTU mismatch, network load, unstable links.

```
ping -M do -s 1472 example.com
tracert example.com
mtr -r -c 100 example.com
```



9 SENIOR SCENARIO

? Requests work from one subnet but time out from another. How do you narrow the failure domain?

INVESTIGATION APPROACH

- 1 Compare DNS resolution from both subnets.
- 2 Test TCP connectivity to the destination IP and port.
- 3 Check routing, NAT and firewall rules on both paths.
- 4 Capture packets or run traceroute/mtr to identify where it fails.
- 5 Check MTU, packet loss and intermediate devices (load balancer, proxy).
- 6 Validate service health and dependency connectivity.

10 KEY THINGS SENIOR ENGINEERS CHECK

- | | |
|-------------------------------------|---------------------------------------|
| ✓ DNS resolution and correct IP | ✓ Connection timeouts and retries |
| ✓ Network path and routing | ✓ MTU and packet loss |
| ✓ Firewall and security group rules | ✓ Application and dependency health |
| ✓ Load balancer and proxy health | ✓ Comparing working vs failing path |
| ✓ TLS handshake and certificate | ✓ Recent changes in network or config |



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

GIT STRATEGY AND RELEASE ENGINEERING

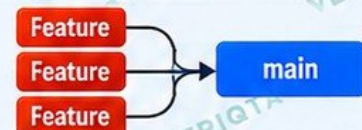
SAFE DELIVERY, STABLE RELEASES, LOWER RISK



1 BRANCHING STRATEGY

TRUNK-BASED DEVELOPMENT

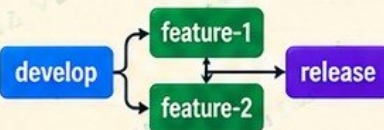
- Short-lived feature branches
- Small, frequent merges
- Continuous integration
- Lower merge risk
- Faster feedback
- Better for large teams



Preferred for most modern DevOps and platform teams.

LONGER-LIVED BRANCHES

- Feature or release branches
- Larger, less frequent merges
- Higher merge conflicts
- Slower feedback
- Useful for regulated environments
- Requires stronger governance



Use when you need controlled release cycles or long features.

2 PULL REQUEST CONTROLS

- Require code review (at least 2 reviewers).
- Automated checks (build, test, security scan).
- Block merge on failing checks.
- Enforce branch protection rules.
- Use required status checks.
- Restrict who can approve and merge.



Require PR reviews (GitHub example)

Require pull request reviews
Require status checks to pass
Restrict who can push to main

3 RELEASE TAGGING

- Use semantic versioning (v1.2.3).
- Tag the exact commit that is deployed.
- Keep tags immutable.
- Link tags to release notes and artifacts.



Create and push a release tag
`git tag -a v1.2.3 -m "Release v1.2.3"`
`git push origin v1.2.3`

4 REVERT vs ROLLBACK

- Revert: create a new commit that undoes the change (safe, keeps history).
- Rollback: redeploy a previous version (works if you use immutable artifacts).
- Prefer revert for code. Prefer rollback for deployment using artifacts.



Revert a bad commit
`git revert <commit-id>`
Rollback deployment (example)
`kubectl rollout undo deployment/app`

5 ARTIFACT IMMUTABILITY

- Build once, promote the same artifact through environments.
- Do not rebuild for each environment.
- Store artifacts in a registry (OCI, JAR, npm, etc).
- Verify checksums and signatures.



Docker image: immutable tag
JAR: versioned artifact
NPM: versioned package
Keep build metadata and SBOM

6 RELEASE TRACEABILITY

- Link each release to:
 - Commit ID
 - Pull request
 - Build pipeline run
 - Artifact version
 - Deployment environment
 - Release notes
 - Change approvals



7 EMERGENCY CHANGES

- Have a documented process.
- Use smaller, safer changes.
- Require post-change review.
- Monitor closely after deployment.
- Prefer feature flags when possible.
- Communicate to stakeholders.



Emergency hotfix flow
Create hotfix branch → review →
deploy → monitor → post-incident review

8 PROTECTING CRITICAL BRANCHES

- Block force pushes.
- Require PR and status checks.
- Restrict direct pushes to main.
- Use CODEOWNERS.
- Enable signed commits (optional).
- Monitor branch changes.



GitHub branch protection (main)
Require pull request reviews
Require status checks
Restrict who can push
Block force pushes

9 SENIOR SCENARIO



A bad commit has reached production while several newer commits are already merged. What is your safest recovery strategy?

KEY TAKEAWAYS

- ✓ Choose a branching strategy that fits your team and risk profile.
- ✓ Use strong PR controls and automated checks.
- ✓ Keep artifacts immutable and traceable.
- ✓ Plan for fast and safe recovery from bad releases.
- ✓ Protect critical branches and have a clear emergency process.

10 APPROACH AND REASONING

- 1 Assess the blast radius and impact on users.
- 2 Identify the exact commit that introduced the issue.
- 3 Decide between revert or redeploy using a previous artifact.
- 4 Consider dependencies, database changes and compatibility.
- 5 Communicate the plan and execute the safest option.
- 6 Verify the fix and monitor closely.
- 7 Perform a follow-up review to prevent recurrence.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

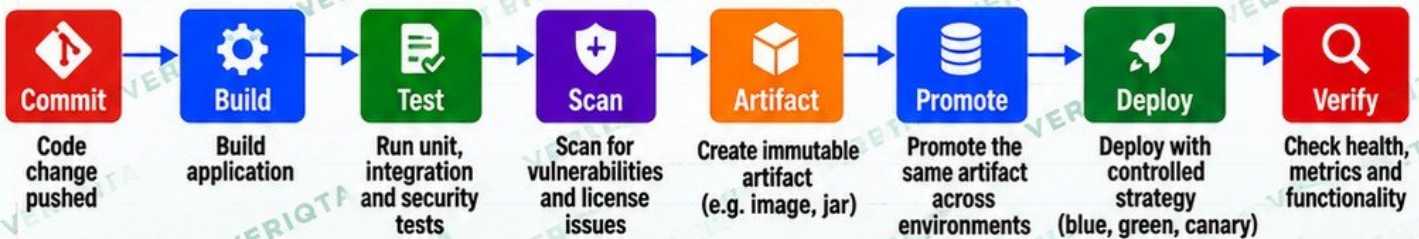


@veriqta

CI/CD PIPELINE ARCHITECTURE

DESIGN A PRODUCTION DELIVERY SYSTEM

1 CI/CD PIPELINE FLOW



2 BUILD ONCE, PROMOTE THE SAME ARTIFACT

- Build the artifact one time and promote it.
- Avoid rebuilding for each environment.
- Ensures consistency and traceability.
- Reduces environment drift and “works on staging but not in production” issues.

Example: build and push Docker image once
`docker build -t app:1.2.3`
`docker push registry/app:1.2.3`

Use the same image in all environments



3 PIPELINE ISOLATION

- Separate pipelines or stages for different environments.
- Use environment-specific credentials and permissions.
- Prevent accidental production deployments.



4 QUALITY GATES

- Automated tests must pass before deployment.
- Include security scans, license checks and policy enforcement.
- Block deployment on failures.

- ✓ Unit tests
- ✓ Integration tests
- ✓ Security scan (SAST/DAST)
- ✓ Image scan
- ✓ Policy checks
- ✓ Approval (if required)

5 DEPLOYMENT APPROVALS

- Require approval for staging and production.
- Use manual or automated approvals based on risk.
- Include change summary and test results.

Example: GitHub Actions environment jobs:
 deploy:
 environment: production
 needs: test
 steps: ...



6 SECRETS AND CREDENTIALS

- Never store secrets in code.
- Use secret management tools (e.g. Vault, AWS Secrets Manager).
- Use short-lived credentials.
- Rotate regularly.



Example: use secrets from environment
`- name: Login to registry`
`run: echo "${ secrets.REGISTRY_TOKEN }" |`
`docker login --username user \`
`--password-stdin registry.example.com`

7 PROVENANCE AND REPRODUCIBILITY

- Track source commit, build metadata and dependencies.
- Use signed artifacts and checksums.
- Ensure builds are reproducible.

Example: record metadata
`git rev-parse HEAD > build.txt`
`docker build --label commit=$(git rev-parse HEAD) --label version=1.2.3`



8 FAILURE HANDLING

- Fail fast on test or security failures.
- Provide clear logs and notifications.
- Automatically rollback when possible.
- Make failures visible to the team.

Example: deploy with automatic rollback
`kubectl apply -f deployment.yaml`
`kubectl rollout status deployment/app \`
`--timeout=5m || kubectl rollout undo \`
`deployment/app`



9 POST-DEPLOYMENT VERIFICATION

- Check application health endpoints.
- Verify key metrics and error rates.
- Run smoke tests.
- Confirm dependencies are healthy.

Example: health check and smoke test
`curl -f https://app.example.com/health`
`curl -f https://app.example.com/api/status`
 # Check metrics (Prometheus/Grafana)



10 SENIOR SCENARIO

? Your organization rebuilds an application separately for staging and production. What risk does that introduce?

KEY POINTS IN YOUR ANSWER

- 1 Builds can produce different artifacts.
- 2 Environment drift and inconsistent behavior.
- 3 Harder to reproduce and debug issues.
- 4 Increases risk of “works in staging, fails in prod”.
- 5 Prefer build once, promote the same artifact with traceability and verification.

ADDITIONAL CONSIDERATIONS

- ✓ Branch protection rules
- ✓ Artifact storage and retention
- ✓ Release notes and change tracking
- ✓ Monitoring and alerting after deployment
- ✓ Disaster recovery for failed releases
- ✓ Least privilege access for pipelines



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

DEPLOYMENT STRATEGIES AND ROLLBACK DECISIONS

CHOOSE THE RIGHT STRATEGY FOR SAFE, RELIABLE RELEASES

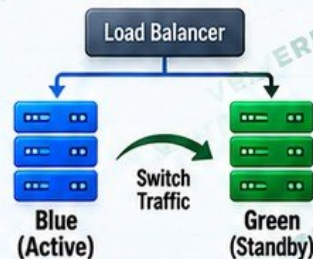
1 ROLLING DEPLOYMENT

- Update instances gradually.
- No downtime.
- Simple and commonly used.
- Risk: both old and new versions run at the same time.



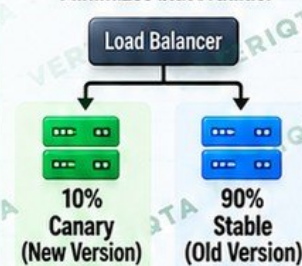
2 BLUE-GREEN DEPLOYMENT

- Two identical environments.
- Switch traffic from blue to green when ready.
- Fast rollback (switch back).
- Higher cost (double resources).



3 CANARY DEPLOYMENT

- Release to a small percentage of users first.
- Monitor real user metrics.
- Gradually increase traffic if healthy.
- Minimizes blast radius.



4 FEATURE FLAGS

- Deploy code but keep feature off.
- Enable for a small group of users.
- No redeploy required to disable.
- Useful for gradual rollout and A/B testing.



5 KEY CONSIDERATIONS

- Blast radius – how many users are affected if something goes wrong.
- Traffic shifting – how traffic is routed and controlled.
- Compatibility – new version must be compatible with dependencies.
- Database migrations – handle schema changes safely.
- Rollback limitations – some changes cannot be easily rolled back.
- Automated verification – health checks, metrics and tests.
- Deployment health signals – error rate, latency, saturation and business metrics.

6 STRATEGY COMPARISON

Feature	Rolling	Blue-Green	Canary	Feature Flags
Downtime	No	No	No	No
Blast Radius	Medium	Low	Very Low	Very Low
Rollback Speed	Medium	Fast	Fast	Instant (disable flag)
Resource Cost	Normal	High	Normal	Normal
Complexity	Low	Medium	Medium	Low
Best For	General use	High availability	Reduce risk	Gradual feature release

7 DATABASE MIGRATIONS

- Use backward compatible changes.
- Apply schema changes before or after deployment based on strategy.
- Consider feature flags for code that depends on new schema.
- Test migrations on a copy of production data.
- Have a rollback plan (or a forward-fix plan).



8 AUTOMATED VERIFICATION

- Run health checks (liveness/readiness).
- Monitor key metrics (error rate, latency, CPU, memory).
- Run synthetic and real user tests.
- Validate business metrics (e.g. checkout success rate).
- Automatically stop rollout if thresholds are breached.



Example: Kubernetes rollout check
`kubectl rollout status deployment/app`
`kubectl rollout history deployment/app`

9 ROLLBACK DECISIONS

- Have a clear rollback strategy before deployment.
- Keep previous version available.
- Use immutable artifacts and versioned releases.
- Know what can and cannot be rolled back (e.g. database changes).
- Practice rollback regularly.



10 SENIOR SCENARIO



A canary has normal CPU and memory but checkout errors increase by 2%. Continue or stop, and what evidence do you inspect?

11 INVESTIGATION CHECKLIST

- 1 Check error rate by endpoint (is it limited to checkout?)
- 2 Compare canary vs stable metrics.
- 3 Check logs, traces and correlation IDs.
- 4 Look for recent changes in dependencies.
- 5 Validate database and external service health.
- 6 Check user impact and business metrics.
- 7 Decide to continue, pause or rollback based on risk.

12 KEY TAKEAWAYS

- ✓ Match the deployment strategy to your risk profile.
- ✓ Minimize blast radius and have a rollback plan.
- ✓ Use automated verification and real user metrics.

- ✓ Keep artifacts immutable and traceable.
- ✓ Handle database changes carefully.
- ✓ Always monitor business metrics, not just infrastructure metrics.

CONTAINERS UNDER PRODUCTION PRESSURE

GO BEYOND DOCKERFILE QUESTIONS



1 IMAGE LAYERS

- Images are built in layers.
- Each layer is immutable.
- Layers are cached to speed up builds.
- Smaller images reduce attack surface.
- Use multi-stage builds.



```
# Example Dockerfile layers
FROM python:3.12-slim
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY app/ /app
```

2 CONTAINER LIFECYCLE

- Created from an image.
- Runs a main process (PID 1).
- Can be stopped, restarted or removed.
- Use proper signal handling (SIGTERM).
- Design for graceful shutdown.

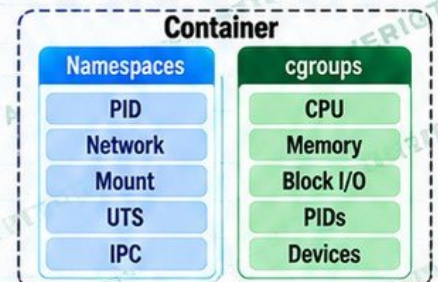


Common commands

```
docker create
docker start
docker stop
docker restart
docker rm
```

3 NAMESPACES AND CGROUPS

- Namespaces isolate the container (processes, network, filesystem, etc).
- cgroups control resource usage (CPU, memory, I/O, etc).
- Provide isolation and prevent one container from affecting others.



4 CPU LIMITS

- Limit how much CPU a container can use.
- Prevents one container from starving others.
- Use requests and limits in Kubernetes.

```
# Docker run example
docker run -d \
  --cpus="0.5" \
  --name app \
  myapp:latest
```



5 MEMORY LIMITS AND OOM BEHAVIOR

- Set memory limits to prevent excessive usage.
- If a container exceeds its limit, it can be killed by the OOM killer (exit code 137).
- Monitor memory usage and set realistic limits.

```
# Docker run example
docker run -d \
  --memory="512m" \
  --memory-swap="512m" \
  myapp:latest
```



6 CONTAINER NETWORKING

- Containers have their own network namespace.
- Communicate using bridge, host, or overlay networks.
- Use DNS for service discovery.
- Control traffic with network policies (in Kubernetes).



```
# Create a bridge network
docker network create app-net
docker run --network app-net myapp
```

7 PERSISTENT DATA

- Containers are ephemeral.
- Use volumes for persistent data.
- Options: bind mount, named volume, or external storage.

```
# Docker volume example
docker volume create db-data
docker run -d \
  -v db-data:/var/lib/mysql \
  mysql:8
```



8 RUNTIME SECURITY

- Run containers as non-root users.
- Drop unnecessary capabilities.
- Use read-only filesystems.
- Scan images for vulnerabilities.
- Use seccomp, AppArmor, or SELinux.

```
# Run as non-root user
docker run -d \
  --user 1000:1000 \
  --read-only \
  --cap-drop=ALL \
  myapp:latest
```



9 IMAGE SUPPLY CHAIN

- Use trusted base images.
- Scan for vulnerabilities (SAST/DAST).
- Sign images (e.g. cosign).
- Verify image provenance (SBOM).
- Pull from trusted registries only.



10 KEY COMMANDS FOR INVESTIGATION

Purpose	Command
Check container status	<code>docker ps -a</code>
View logs	<code>docker logs <container></code>
Inspect container details	<code>docker inspect <container></code>
Check resource usage	<code>docker stats <container></code>
Enter container shell	<code>docker exec -it <container> sh</code>
Check processes	<code>docker top <container></code>
Check events	<code>docker events</code>
Check image details	<code>docker image inspect <image></code>
Analyze disk usage	<code>docker system df</code>

11 SENIOR SCENARIO

? A container is repeatedly killed even though the host has plenty of free memory. Explain what you investigate.

INVESTIGATION APPROACH

- 1 Check container memory limits and usage.
- 2 Look for OOMKilled events (exit code 137).
- 3 Inspect application memory behavior and leaks.
- 4 Verify cgroup memory constraints.
- 5 Check logs and container events.
- 6 Review recent changes (image, config, dependencies).
- 7 Check if multiple containers are competing for resources.

12 KEY TAKEAWAYS

- ✓ Understand how containers isolate and control resources.
- ✓ Set appropriate CPU and memory limits.
- ✓ Monitor and alert on container resource usage.
- ✓ Use volumes for persistent data.
- ✓ Follow container security best practices.
- ✓ Use signed and scanned images.
- ✓ Have a clear troubleshooting process for production issues.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

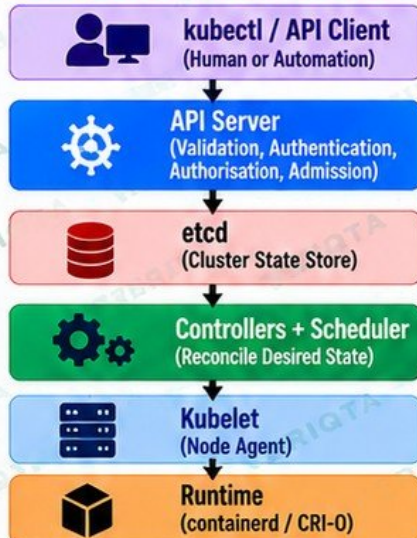


@veriqta

KUBERNETES CONTROL PLANE AND RECONCILIATION

HOW KUBERNETES KEEPS YOUR DESIRED STATE RUNNING

1 CONTROL PLANE ARCHITECTURE



2 DESIRED VS OBSERVED STATE

- You declare the desired state in a YAML file (e.g. 6 replicas).
- Kubernetes continuously compares the desired state with the actual (observed) state.
- If they do not match, controllers take action to reconcile.



3 KEY COMPONENTS

API Server	Central entry point. Validates, authenticates, applies admission controls.
etcd	Distributed key-value store that stores the cluster state.
Controllers	Continuously watch the cluster and reconcile differences (e.g. Deployment, ReplicaSet).
Scheduler	Decides which node a Pod should run on based on resources, affinity, taints and constraints.
Kubelet	Runs on each node. Ensures containers are running as specified.
Runtime	Pulls images and runs containers (e.g. containerd, CRI-O).

4 CONTROLLERS AND RECONCILIATION

- Controllers watch for changes in the API Server.
- Compare desired vs observed state.
- Take action to make them match.
- Examples: Deployment, ReplicaSet, StatefulSet, DaemonSet, Job, CronJob.

Deployment controller workflow

- 1 Detects missing Pods (desired: 6, observed: 4)
- 2 Creates new ReplicaSets or Pods
- 3 Monitors until desired state is achieved
- 4 Repeats continuously

5 SCHEDULER DECISIONS

- Selects the best node for a Pod.
- Considers available resources (CPU, memory, storage).
- Uses constraints (node selectors, affinity/anti-affinity, taints/tolerations).
- Pod remains in Pending if no suitable node is found.

Scheduler factors

- Resource requests and limits
- Node capacity and availability
- Affinity and anti-affinity rules
- Taints and tolerations
- Topology spread constraints

6 RECONCILIATION LOOP



7 CONTROL-PLANE FAILURE IMPLICATIONS

- API Server down:** cluster cannot be managed, controllers cannot receive or process requests.
- etcd issues:** loss of cluster state, data corruption, or read/write failures can stop the entire cluster.
- Controller failure:** desired state may not be reconciled, leading to drift.
- Scheduler failure:** new Pods cannot be scheduled (existing Pods continue running).
- Kubelet failure:** Pod management on affected node stops.
- Network or certificate issues:** components may not communicate.

8 SENIOR SCENARIO

? A Deployment requests six replicas, but only four Pods exist. How do you determine which component is preventing convergence?

INVESTIGATION APPROACH

- 1 Check Deployment and ReplicaSet status.
- 2 Verify controller logs and events.
- 3 Check for scheduler issues (Pending Pods).
- 4 Inspect node resources, taints, and capacity.
- 5 Look at kubelet and runtime logs on nodes.
- 6 Check API Server and etcd for errors.
- 7 Validate RBAC, quotas, and admission policies.

9 USEFUL COMMANDS

<code>kubectl get deploy</code>	Check desired/observed
<code>kubectl get rs</code>	Check ReplicaSets
<code>kubectl get pods -o wide</code>	Check Pod status and nodes
<code>kubectl describe pod <pod></code>	View events and reasons
<code>kubectl get events --sort-by=.lastTimestamp</code>	Check recent events
<code>kubectl logs -n kube-system <controller-pod></code>	Check controller logs
<code>kubectl get nodes</code>	Check node resources
<code>kubectl describe node <node></code>	View node details
<code>kubectl get componentstatuses</code>	Check control-plane health

10 KEY TAKEAWAYS

- Understand how each control-plane component works.
- Know how reconciliation keeps the desired state running.
- Be able to troubleshoot when the actual state does not match the desired state.
- Control-plane failures have different impacts on the cluster.
- Use events, logs and metrics to quickly identify the failure domain.

11 COMMON ISSUES THAT PREVENT CONVERGENCE

- Insufficient node resources (CPU, memory, storage).
- Pod cannot be scheduled due to taints, affinity or topology rules.
- Image pull failures.
- CrashLoopBackOff (container exits repeatedly).
- Readiness or liveness probe failures.
- RBAC, quota or admission policy rejections.
- Controller not running or not processing events.
- API Server or etcd errors.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

KUBERNETES SCHEDULING, RESOURCES AND NODE PRESSURE

HOW PODS GET SCHEDULED AND WHY NODES REJECT THEM

1 REQUESTS VS LIMITS

- Requests: guaranteed resources used for scheduling.
- Limits: maximum resources a container can use.
- Scheduler uses requests, not limits.
- If limits are not set, a container can use more than expected and cause noisy neighbor issues.

resources:
requests:
cpu: "500m"
memory: "512Mi"
limits:
cpu: "1"
memory: "1Gi"



2 QoS CLASSES

- Guaranteed: requests = limits for both CPU and memory.
- Burstable: requests < limits.
- BestEffort: no requests or limits.
- QoS affects eviction priority during node pressure.

QoS Class	CPU	Memory	Eviction Priority
Guaranteed	req = limit	req = limit	Lowest
Burstable	req < limit	req < limit	Medium
BestEffort	not set	not set	Highest

3 CPU THROTTLING

- Occurs when a container exceeds its CPU limit.
- Kernel restricts CPU usage (CFS throttling).
- Can cause performance issues without high CPU usage on the node.

Check throttling metrics:
kubectll top pod <pod>
cat /sys/fs/cgroup/cpu.stat



4 OOMKILLED

- Occurs when a container exceeds its memory limit.
- Container is terminated with exit code 137.
- Check logs and events to confirm.

Check if a container was OOMKilled:
kubectll describe pod <pod>
kubectll logs <pod> --previous



5 EVICTION AND NODE PRESSURE

- Node pressure occurs when a node is low on:
 - Memory
 - Disk space (ephemeral storage)
 - Inodes
 - PID count
- Kubelet evicts Pods based on QoS, usage and eviction thresholds.

Check node conditions:
kubectll describe node <node>
kubectll get events --sort-by=.lastTimestamp



6 TAINTS AND TOLERATIONS

- Taints prevent Pods from being scheduled on nodes.
- Tolerations allow specific Pods to be scheduled on tainted nodes.

Example taint:
kubectll taint node node1 \
key=value:NoSchedule

Example toleration (pod spec):
tolerations:
- key: "key"
operator: "Equal"
value: "value"
effect: "NoSchedule"

7 AFFINITY AND TOPOLOGY SPREAD

- Node affinity: schedule Pods on specific nodes (e.g. instance type, labels).
- Pod affinity/anti-affinity: control where Pods run based on other Pods.
- Topology spread: distribute Pods across zones, nodes, or other topology domains.



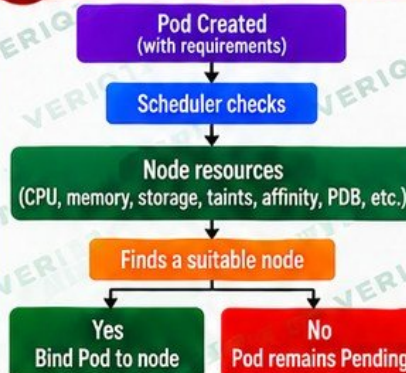
8 PODDISRUPTIONBUDGETS (PDB)

- Limits the number of Pods that can be unavailable during voluntary disruptions (e.g. node upgrades).
- Helps maintain application availability.

Example PDB:
apiVersion: policy/v1
kind: PodDisruptionBudget
spec:
minAvailable: 2
selector:
matchLabels:
app: myapp



9 SCHEDULING DECISION FLOW



10 SENIOR SCENARIO

? Pods are Pending while cluster CPU utilization appears low. Why can both observations be true?

POSSIBLE REASONS

- Insufficient CPU/memory requests on available nodes.
- Nodes have taints that the Pods do not tolerate.
- Pod affinity / anti-affinity rules prevent scheduling.
- PDB or topology spread constraints cannot be satisfied.
- Node resources like ephemeral storage or inodes are exhausted.
- Nodes are cordoned or unschedulable.
- Requested node selectors or labels do not match any node.

11 USEFUL COMMANDS

Task	Command
Check pending Pods	kubectll get pods -A --field-selector=status.phase=Pending
Describe Pod (events)	kubectll describe pod <pod>
Check node resources	kubectll describe node <node>
View taints	kubectll get nodes -o custom-columns=NAME:.metadata.name,TAINTS:.spec.taints
Check PDBs	kubectll get pdb -A
Check scheduler logs	kubectll logs -n kube-system deployment/kube-scheduler

12 KEY TAKEAWAYS

- Requests determine scheduling, limits control usage.
- QoS affects eviction priority.
- Node pressure can evict Pods even when cluster CPU looks low.
- Taints, affinity and topology spread can prevent scheduling.
- PDBs can block voluntary disruptions.
- Always check events, node conditions and scheduler logs when Pods are Pending.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

KUBERNETES NETWORKING AND SERVICE FAILURE

FOLLOW TRAFFIC THROUGH KUBERNETES

1 TRAFFIC FLOW ARCHITECTURE



2 SERVICE DISCOVERY

- Services provide stable DNS names and virtual IPs.
- kube-proxy or CNI handles service routing.
- EndpointSlice tracks healthy endpoints.
- Supports different service types (ClusterIP, NodePort, LoadBalancer).

Example: check service and endpoints

```
kubectl get svc
kubectl get endpointslice
kubectl describe svc <service>
```



3 COREDNS

- Resolves service names to ClusterIP.
- Uses cluster DNS configuration.
- Issues can cause timeouts even when Pods are running.
- Check CoreDNS logs and configuration.

Example: test DNS resolution

```
kubectl run -it --rm dnsutils \
--image=busybox -- nslookup \
myservice.default.svc.cluster.local
```



4 CNI (CONTAINER NETWORK INTERFACE)

- Provides Pod networking within the cluster.
- Handles cross-node communication.
- Different CNIs (Calico, Cilium, Flannel, etc.) have different features.
- Issues can cause Pod-to-Pod or Service routing failures.

Example: check CNI and node networking

```
kubectl get pods -n kube-system
kubectl describe pod <cni-pod>
kubectl get nodes -o wide
```



5 NETWORKPOLICY

- Controls inbound and outbound traffic to Pods.
- Can block traffic even when Pods are Ready.
- Use labels to allow required traffic.
- Check for policies that might be too restrictive.

Example: view network policies

```
kubectl get networkpolicy
kubectl describe networkpolicy
kubectl get pods --show-labels
```



6 SERVICE ROUTING

- Service selects Pods using labels.
- kube-proxy (iptables or IPVS) routes traffic to healthy endpoints.
- EndpointSlice only includes ready Pods.
- Misconfigured selectors or no ready endpoints cause timeouts.

Example: check service routing

```
kubectl get svc <service> -o yaml
kubectl get endpointslice -l kubernetes.io/
service-name=<service>
kubectl get pods -l <selector-labels>
```



7 INGRESS

- Handles external HTTP/S traffic.
- Routes to services based on host/path rules.
- Uses ingress controller (e.g. NGINX, AWS ALB, Traefik).
- Check ingress, controller logs and configuration.

Example: check ingress

```
kubectl get ingress
kubectl describe ingress
kubectl logs -n ingress-nginx
<ingress-controller-pod>
```



8 ENDPOINT HEALTH

- Only ready Pods are added to EndpointSlice.
- If readiness probes fail, Pods are not included.
- Check Pod health, readiness/liveness probes, and application logs.

Example: check endpoint health

```
kubectl get endpointslice -l
kubernetes.io/service-name=<service>
kubectl describe pod <pod>
kubectl logs <pod>
```



9 CROSS-NODE COMMUNICATION

- Pods on different nodes communicate via the CNI.
- Check node networking, routes and firewall rules.
- Ensure no network policies or security groups are blocking traffic.

Example: test connectivity between Pods

```
kubectl exec -it <pod> -- ping <pod-ip>
kubectl exec -it <pod> -- curl <service>
traceroute <pod-ip>
```



10 SENIOR SCENARIO

- Pods are Ready and the Service exists, but users receive timeouts. Build the investigation path.

INVESTIGATION APPROACH

- Verify the issue from the client (external and internal).
- Check ingress / load balancer health and logs.
- Confirm service, endpoints and endpoint readiness.
- Test DNS resolution (CoreDNS).
- Check network policies, security groups and firewalls.
- Test Pod-to-Pod and cross-node connectivity.
- Review application logs, readiness probes and errors.
- Identify where the latency or timeout is occurring.

11 USEFUL COMMANDS

Purpose	Command
Check service	<code>kubectl get svc <service> -o wide</code>
Check endpoints	<code>kubectl get endpointslice</code>
Describe ingress	<code>kubectl describe ingress</code>
Check CoreDNS logs	<code>kubectl logs -n kube-system -l k8s-app=kube-dns</code>
Check network policies	<code>kubectl get networkpolicy -A</code>
Test from Pod	<code>kubectl exec -it <pod> -- curl <service></code>
Check CNI pods	<code>kubectl get pods -n kube-system</code>
Check node networking	<code>kubectl get nodes -o wide</code>

12 KEY TAKEAWAYS

- Understand the full traffic path from client to Pod. Service, EndpointSlice and readiness must align.
- DNS, CNI and network policies can all cause timeouts.
- Ingress and load balancers must be healthy and correctly configured.
- Always check both cluster and node-level networking.
- Use a systematic approach to isolate the failure domain.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

KUBERNETES PRODUCTION INCIDENT

INTERVIEW CASE FILE: DEPLOYMENT SUCCEEDED. PRODUCTION FAILED.



1 INCIDENT OVERVIEW

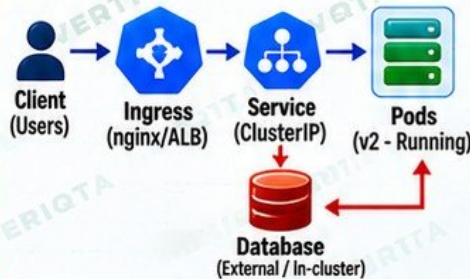
Title: Deployment succeeded. Production failed.

Environment: Kubernetes (production)

Impact: Users experiencing failures, increasing latency.

Scenario: A new version was deployed.
Deployment completed successfully and all Pods are Running with Readiness green, but the application is failing for users.

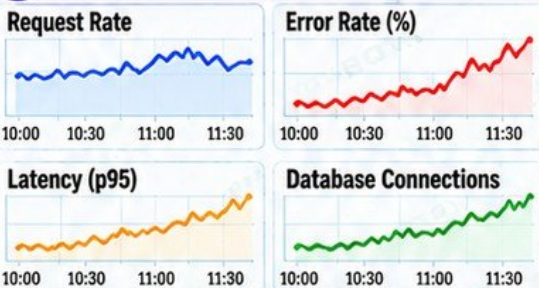
2 ARCHITECTURE



3 OBSERVED EVIDENCE

- ✓ Pods are Running.
- ✓ Readiness probes are green.
- ↑ Error rate is rising.
- ↑ Database connections are climbing.
- ↑ Latency is increasing.

4 METRICS SNAPSHOT (EXAMPLE)



5 RECENT CHANGES (CHECK)

- New application version deployed.
- Configuration changes (env vars, ConfigMap, Secret).
- Database migration or schema change.
- Ingress / Service update.
- Resource limits or autoscaling changes.
- Dependency version update.
- Feature flag enabled.

6 BLAST RADIUS (DETERMINE)

- Is the issue affecting all users or a subset?
- Is it limited to a specific service or namespace?
- Is one region/zone affected?
- Are background jobs also impacted?
- Are other applications that use the same database affected?
- Internal traffic vs external traffic.
- Check if previous version is also impacted.

7 RELEVANT TELEMETRY (COLLECT)

- Application logs (current and previous version).
- Pod logs and events.
- Ingress/controller logs.
- Service and EndpointSlice information.
- Database metrics (connections, slow queries).
- Cluster metrics (CPU, memory, network, etc.).
- Tracing (if available).
- Recent changes from CI/CD, Git, or deployment tools.



8 HYPOTHESES (EXAMPLES)

- New version has a performance issue (e.g. inefficient query).
- Database connection leak or pool misconfiguration.
- External dependency is slow or failing.
- Configuration change caused higher load.
- Resource limits causing throttling or restarts.
- Readiness probe is too simple and not detecting real issues.
- Feature flag enabled and causing unexpected behavior.
- Network or DNS issue to database or external service.



9 IMMEDIATE MITIGATION (EXAMPLES)

- Roll back to previous version (if safe).
- Disable the new feature via feature flag.
- Scale up replicas.
- Increase database connection pool limit (temporary).
- Enable rate limiting or circuit breaker.
- Reroute traffic (e.g. to healthy region).



10 ROOT CAUSE (FIND AND EXPLAIN)

- Identify the exact cause (e.g. connection leak, slow query, misconfiguration).
- Explain why it was not detected before deployment.
- Explain how it led to the observed symptoms (error rate, latency, DB connections).



11 VERIFICATION (CONFIRM)

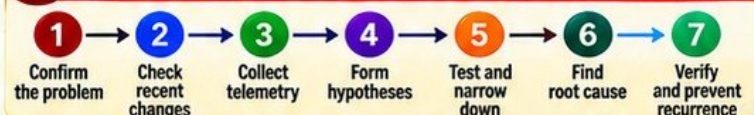
- Error rate returns to normal.
- Latency back to normal.
- Database connections stabilize.
- Users can successfully complete their requests.
- Monitor for a period to ensure no recurrence.



12 USEFUL KUBECTL COMMANDS

Purpose	Command
Check deployment status	<code>kubectl get deploy -n <ns></code>
Check pods	<code>kubectl get pods -n <ns> -o wide</code>
Describe pod	<code>kubectl describe pod <pod> -n <ns></code>
Check pod logs	<code>kubectl logs <pod> -n <ns> --previous</code>
Check events	<code>kubectl get events -n <ns> --sort-by=.lastTimestamp</code>
Check service and endpoints	<code>kubectl get svc,ep,endpointslice -n <ns></code>
Check ingress	<code>kubectl get ingress -n <ns></code>
Check resource usage	<code>kubectl top pods -n <ns></code>
Check node status	<code>kubectl get nodes -o wide</code>
Check recent changes	<code>kubectl rollout history deploy/<name> -n <ns></code>

13 INVESTIGATION FLOW (THINK LIKE AN SRE)



14 KEY TAKEAWAYS FOR INTERVIEWS

- ✓ Always investigate beyond pod status and readiness.
- ✓ Follow the full request path and check dependencies.
- ✓ Use real telemetry (logs, metrics, traces) to form and test hypotheses.
- ✓ Communicate your reasoning, not just commands.
- ✓ Explain trade-offs for mitigation and prevention.
- ✓ Show a structured and methodical approach.



INFRASTRUCTURE AS CODE AND TERRAFORM AT SCALE

BUILD, MANAGE AND OPERATE INFRASTRUCTURE SAFELY WITH TERRAFORM

1 WHAT IS IaC AND TERRAFORM?

- Infrastructure as Code (IaC) lets you define, version and manage infrastructure using code.
- Terraform is a declarative IaC tool that provisions and manages cloud and on-prem resources.
- It uses a desired state model to create, update and destroy resources safely.

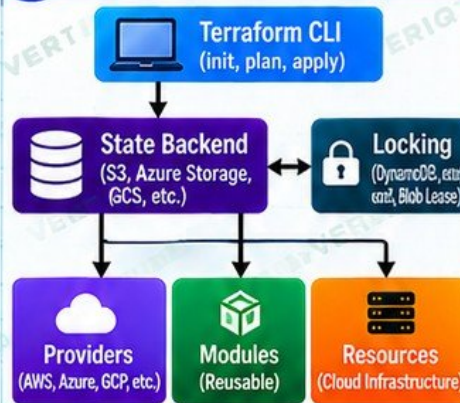


Terraform

Key benefits:

- Consistent infrastructure
- Repeatable deployments
- Version control
- Better collaboration
- Reduced manual errors

2 TERRAFORM ARCHITECTURE



3 STATE AND REMOTE BACKENDS

- State stores the current infrastructure and maps it to your configuration.
- Use a remote backend (e.g. S3, Azure Storage, GCS) for team collaboration.
- Enable state locking to prevent concurrent changes.
- Never store state files locally in teams or production environments.

Example: S3 backend with locking

```

terraform {
  backend "s3" {
    bucket = "veriqta-terraform-state"
    key    = "prod/terraform.tfstate"
    region = "us-east-1"
    dynamodb_table = "terraform-locks"
  }
}
  
```

4 MODULES AND REUSABILITY

- Modules help you reuse and standardize infrastructure.
- Use versioned, tested modules.
- Keep modules small and focused.
- Use a clear module structure with inputs and outputs.

Example: module usage

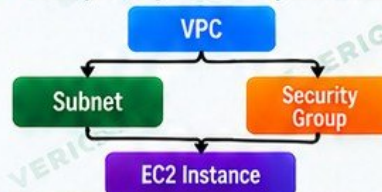
```

module "vpc" {
  source = "git::https://github.com/org/vpc.git"
  version = "1.2.0"
  cidr_block = "10.0.0.0/16"
}
  
```



5 DEPENDENCY GRAPH

- Terraform builds a dependency graph to understand the order of resource creation.
- Implicit dependencies are inferred automatically.
- Use explicit depends_on only when needed.



6 PLAN REVIEW

- Always review the terraform plan output.
- Look for creates, updates and replaces.
- Understand why a resource will be replaced.
- Check for unintended changes.

\$ terraform plan

```

+ create (will be created)
~ update (will be updated in-place)
-/+ replace (will be destroyed and created)
- destroy (will be destroyed)
  
```

7 DRIFT DETECTION

- Drift occurs when real infrastructure differs from the Terraform state.
- Use terraform plan regularly to detect drift.
- Avoid making manual changes in the cloud console.

\$ terraform plan

```

~ resource "aws_instance" "app" {
  instance_type = "t3.medium" -> "t3.large"
}
  
```



8 IMPORTS

- Use terraform import to bring existing resources under Terraform management.
- Useful for brownfield environments.

```

# Import an existing AWS instance
terraform import \
  aws_instance.app \
  i-0abc123def456
  
```



9 REPLACEMENT SCENARIOS

- Some changes require resource replacement (e.g. immutable fields).
- Understand the impact before approving.
- Use lifecycle rules (e.g. create_before_destroy) when appropriate.

```

resource "aws_instance" "app" {
  ami = "ami-123456"
  instance_type = "t3.medium"
  lifecycle {
    create_before_destroy = true
  }
}
  
```



10 SENSITIVE VALUES

- Use sensitive variables for passwords, keys and other secrets.
- Avoid hardcoding sensitive values.
- Use environment variables or a secrets manager.

```

variable "db_password" {
  type = string
  sensitive = true
}
  
```



11 ENVIRONMENT ISOLATION

- Use separate workspaces or separate state files for dev, staging and production.
- Apply least privilege access.
- Use different cloud accounts or subscriptions where possible.



12 STATE RECOVERY

- Keep regular backups of your state.
- If state is lost or corrupted, restore from backup.
- Use remote backends with versioning.
- Document the recovery process.



13 SENIOR SCENARIO

? terraform plan proposes replacing a critical production resource. What do you do before approving it?

14 KEY INVESTIGATION STEPS

- Review the plan output in detail.
- Understand why the resource is being replaced.
- Check recent changes in code, variables and provider versions.
- Validate impact (downtime, data loss, dependencies).
- Test in a non-production environment if possible.
- Consider lifecycle rules or configuration changes.
- Get peer review and approval before applying.

15 KEY TAKEAWAYS

- Use remote state with locking.
- Always review plans carefully.
- Detect and manage drift.
- Use reusable, versioned modules.
- Protect sensitive values.
- Isolate environments.
- Have a state recovery plan.
- Understand replacement risks.

CLOUD ARCHITECTURE AND HIGH AVAILABILITY

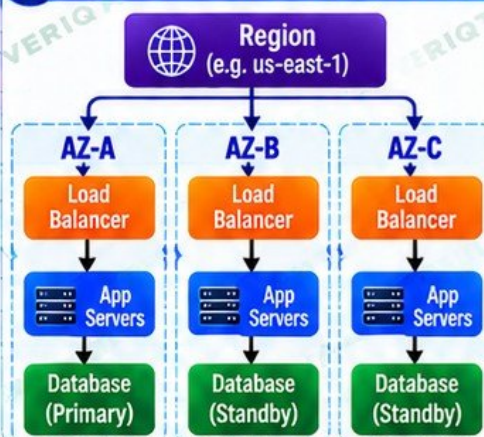
DESIGN RESILIENT, SCALABLE AND COST-EFFECTIVE CLOUD ARCHITECTURES

1 REGIONS VS AVAILABILITY ZONES

- A Region is a separate geographic location with independent infrastructure.
- An Availability Zone (AZ) is an isolated data center within a Region.
- AZs have separate power, networking and cooling to reduce the risk of a single point of failure.
- Design for multi-AZ within a Region for high availability.



2 CLOUD ARCHITECTURE (MULTI-AZ)



3 LOAD BALANCING

- Distributes traffic across multiple instances or AZs.
- Supports health checks and automatic failover.
- Can be public (internet-facing) or internal (private).
- Improves availability and scalability.



4 PRIVATE NETWORKING

- Use VPC (Virtual Private Cloud) to isolate resources.
- Use subnets (public and private).
- Use route tables, security groups and network ACLs.
- Keep databases and internal services in private subnets.



5 NAT (NETWORK ADDRESS TRANSLATION)

- Allows resources in private subnets to access the internet (e.g. for updates).
- Use a NAT Gateway (managed) or NAT Instance.
- Deploy in multiple AZs for high availability.
- Avoid single NAT gateway as a single point of failure.



6 FAULT DOMAINS

- Cloud providers isolate failures at multiple levels (hardware, power, networking, AZ).
- Design for failure by removing single points of failure.
- Use multi-AZ and, if required, multi-Region for critical systems.



7 MANAGED SERVICES

- Use managed services (e.g. RDS, S3, DynamoDB, GKE, EKS, AKS) for higher availability and reduced operational effort.
- Understand the service SLA and availability design.
- Use multi-AZ or multi-Region options where available.



8 QUOTAS AND LIMITS

- Cloud accounts have resource quotas (e.g. vCPUs, IP addresses, load balancers).
- Monitor usage and request limit increases early.
- Design with quotas in mind to avoid deployment failures.



9 REDUNDANCY

- Remove single points of failure.
- Use multiple AZs, multiple subnets and multiple instances.
- Replicate data where required.
- Design for component and AZ failure.



10 BACKUP STRATEGY

- Back up critical data regularly.
- Store backups in a different AZ or Region.
- Test restore procedures.
- Use automated and encrypted backups.



11 DISASTER RECOVERY (DR)

- Plan for region-level failures.
- Use multi-Region deployment for critical workloads.
- Define Recovery Time Objective (RTO) and Recovery Point Objective (RPO).
- Regularly test DR procedures.



12 COST VS RESILIENCE

- Higher availability and redundancy increase costs.
- Use the right level of resilience based on business requirements.
- Consider managed services, autoscaling and right-sizing to optimize costs.



13 SENIOR SCENARIO



The application runs across three Availability Zones, yet one dependency can still take down the entire service. Identify the architectural problem.

14 INVESTIGATION APPROACH

- 1 Identify the dependency (database, cache, queue, external API).
- 2 Check if the dependency is deployed in a single AZ or Region.
- 3 Review the architecture diagram and configuration.
- 4 Check for single points of failure (e.g. single database, single NAT, single load balancer, single external service).
- 5 Verify if there is proper redundancy and failover.
- 6 Recommend architecture changes to eliminate the risk.



15 KEY TAKEAWAYS

- ✓ Use multi-AZ and, where needed, multi-Region.
- ✓ Remove single points of failure.
- ✓ Understand service limits and quotas.
- ✓ Implement proper backup and DR.
- ✓ Balance cost with resilience based on business requirements.

IAM, SECRETS AND PRODUCTION SECURITY

CONTROL ACCESS, PROTECT SECRETS AND BUILD A SECURE PRODUCTION ENVIRONMENT

1 LEAST PRIVILEGE

- Give only the minimum permissions needed to perform a task.
- Use fine-grained permissions (resource, action, condition).
- Avoid using administrator or wildcard permissions.
- Review and remove unused permissions regularly.



Example (AWS IAM policy):

```
{
  "Effect": "Allow",
  "Action": ["s3:GetObject"],
  "Resource": "arn:aws:s3::my-bucket/*"
}
```

2 HUMAN VS WORKLOAD IDENTITY

Human Identity (Users)



- For people (developers, operators, admins).
- Use MFA.
- Short-lived sessions.
- Individual accountability and audit logs.

Examples

IAM user, SSO (OIDC), MFA, role assumption.

Workload Identity (Applications/Services)



- For applications, CI/CD, services, Pods.
- Use short-lived credentials.
- Prefer workload identity (federation) over static keys.

Examples

IAM roles, OIDC federation, Kubernetes service accounts, workload identity.

3 SHORT-LIVED CREDENTIALS

- Use temporary credentials instead of long-lived keys.
- Use IAM roles, OIDC or workload identity federation.
- Set reasonable session duration.
- Automate credential retrieval and rotation.



Example (AWS Assume Role via CLI):

```
aws sts assume-role \
--role-arn arn:aws:iam::123456789012:role/ci-role \
--role-session-name ci-pipeline
```

4 SECRET STORAGE

- Store secrets in a dedicated secrets manager (not in code or Git).
- Use managed services (AWS Secrets Manager, Azure Key Vault, GCP Secret Manager).
- Encrypt secrets at rest and in transit.
- Control access with IAM/RBAC.



Examples

AWS Secrets Manager
Azure Key Vault
GCP Secret Manager
Kubernetes Secrets (for non-sensitive or use external secrets operator)

5 ROTATION

- Rotate secrets regularly.
- Automate rotation where possible.
- Use short-lived credentials to reduce rotation needs.
- Update dependent services automatically and safely.



Rotation options

Automatic rotation (managed service)
Scheduled rotation (scripts/pipelines)
On-demand rotation (incident response)

6 RBAC (ROLE-BASED ACCESS CONTROL)

- Use RBAC to control access to resources (e.g. Kubernetes, cloud services).
- Follow least privilege.
- Use roles and role bindings.
- Regularly review roles and permissions.



Example (Kubernetes Role):

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

7 AUDITABILITY

- Enable and review audit logs.
- Track who accessed what, when and from where.
- Use centralized logging and SIEM.
- Monitor unusual or unauthorized access.



Examples

AWS CloudTrail
Azure Activity Logs
GCP Audit Logs
Kubernetes Audit Logs

8 ENCRYPTION

- Encrypt data at rest (databases, storage, backups, secrets).
- Encrypt data in transit (TLS).
- Use provider-managed or customer-managed keys (KMS).
- Apply encryption to logs and snapshots.



Examples

AWS KMS, Azure Key Vault, GCP KMS
TLS for all traffic
Encrypted EBS/Blob/GCS, database encryption

9 CI/CD CREDENTIALS

- Do not store long-lived credentials in CI/CD.
- Use OIDC federation or short-lived tokens.
- Restrict permissions to only what the pipeline needs.
- Use environment-specific access.
- Rotate and monitor usage.



Best practices

Use OIDC (GitHub Actions, GitLab, etc.)
Use least privilege IAM roles
Separate roles for dev, staging, production
Never hardcode secrets in code or pipeline configs.

10 SUPPLY-CHAIN SECURITY

- Use trusted base images.
- Scan images for vulnerabilities.
- Sign images (e.g. Cosign).
- Verify dependencies (SBOM).
- Use admission policies (e.g. Kubernetes).



Tools and practices

Image scanning (Trivy, Gype)
Image signing (Cosign)
SBOM (Syft)
Policy enforcement (OPA, Kyverno)

11 BREAK-GLASS ACCESS

- Emergency access for critical incidents only.
- Keep it tightly controlled and audited.
- Use time-limited access.
- Require approval and justification.
- Monitor and review usage.



Best practices

Dedicated break-glass role
Short time-to-live (e.g. 1 hour)
MFA and approval workflow
Full audit logging
Regular access review

12 SENIOR SCENARIO

? A CI pipeline uses a long-lived cloud administrator credential. How would you redesign access?

KEY POINTS TO COVER

- 1 Remove long-lived credentials.
- 2 Use OIDC federation or workload identity.
- 3 Create a dedicated role with least privilege.
- 4 Use short-lived, automatically rotated credentials.
- 5 Separate roles per environment (dev, staging, prod).
- 6 Add approval and audit logging.
- 7 Monitor usage and set alerts.
- 8 Validate and test the new approach.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

OBSERVABILITY AND DEBUGGING FROM EVIDENCE

CORRELATE METRICS + LOGS + TRACES + EVENTS + CHANGES TO FIND REAL ISSUES

1 THE BIG PICTURE

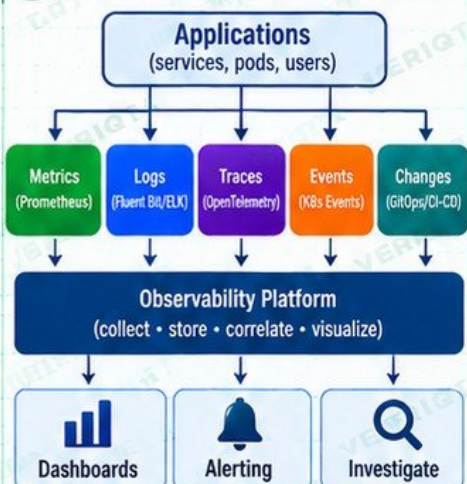
Each signal tells part of the story. Together they give you the full picture.



2 THE SIGNALS & WHAT THEY TELL YOU

	RED	Request rate, error rate, latency, saturation.
	USE	CPU, memory, disk, network, I/O.
	HIGH CARDINALITY	Unique labels, exploding series, storage cost.
	DISTRIBUTED TRACING	Request flow, latency, service dependencies.
	CORRELATION ID	Link logs, traces, and requests across services.
	SLO SIGNALS	Availability, latency, error budget.
	ALERT QUALITY	Right signal, less noise, actionable alerts.
	DASHBOARD LIMITATIONS	Aggregated, delayed, missing context.

3 OBSERVABILITY ARCHITECTURE



4 RED vs USE SIGNALS

RED (Service Health)

- Rate (requests/second)
- Errors (error rate)
- Duration (latency)

USE (Resource Health)

- Utilization (CPU, memory)
- Saturation (disk, network)
- Errors (I/O, node)

Example (Kubernetes)

```
# RED - error rate
rate(http_requests_total{status=~"5.."}[5m])

# USE - CPU usage
rate(container_cpu_usage_seconds_total[5m])
```



5 HIGH CARDINALITY PROBLEMS

- Too many unique values (e.g. user_id, pod_name).
- Increases memory, CPU and storage cost.
- Slows queries and dashboards.
- Can cause data loss or dropped metrics.

Example

Instead of user_id (100k+), use:
user_tier: {"free", "pro", "enterprise"}



user_id
(100k+ series)



user_tier
(3 series)

6 DISTRIBUTED TRACING & CORRELATION IDS

- Tracks a request across multiple services.
- Shows where time is spent.
- Helps find failures and bottlenecks.
- Correlation IDs link logs, traces and metrics.

Example Trace Flow

Frontend → API → Auth → DB

Correlation ID: 9f3a7e2d-4b1c-4e2a-9d8f

Check the same ID in logs, traces and events.



7 SLO SIGNALS & ALERT QUALITY

SLO Signals

- Availability (e.g. 99.9%)
- Latency (e.g. p95 < 200ms)
- Error budget (track burn rate)

Good Alerts

- ✓ Actionable
- ✓ Low noise
- ✓ Clear context
- ✓ Tied to SLOs

Bad Alert Example

- ✗ High CPU on node
→ Too generic. What service? What impact?
- ✓ API error rate > 5% for 5m (SLO breach)
→ Actionable, specific, tied to user impact.



8 DASHBOARD LIMITATIONS

- Shows a snapshot, not the full story.
- Can be delayed or missing data.
- Aggregates multiple issues.
- Can hide failures (e.g. 1% error rate).

What to do

- ✓ Always check raw data (logs, traces).
- ✓ Use time range and correlation IDs.
- ✓ Validate with multiple signals.
- ✓ Don't rely on a single dashboard.



9 SENIOR SCENARIO

? All dashboards are green, but customers report failures. What does that tell you about your observability?

KEY INVESTIGATION STEPS

- 1 Check error rates and latency in raw metrics.
- 2 Search logs for errors (with correlation IDs).
- 3 Review traces for failed requests.
- 4 Check recent changes and deployments.
- 5 Look for issues in external dependencies.
- 6 Validate with user reports and timestamps.

10 QUICK INVESTIGATION CHECKLIST

- ✓ Correlate the five signals (metrics, logs, traces, events, changes).
- ✓ Use correlation IDs across all tools.
- ✓ Validate with raw data (not just dashboards).
- ✓ Check recent changes and dependencies.
- ✓ Look for patterns, not just single points.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

RELIABILITY ENGINEERING, SLOs AND ERROR BUDGETS

BUILD RELIABLE SERVICES, MEASURE WHAT MATTERS AND MAKE BETTER DECISIONS

1 KEY CONCEPTS

- **SLI (Service Level Indicator)**
A measurable signal of service performance.
- **SLO (Service Level Objective)**
Target value for an SLI (e.g. 99.9% availability).
- **SLA (Service Level Agreement)**
Contractual commitment, often with penalties.
- **Availability**
Percentage of successful requests.
- **Error Budget**
The allowed amount of failure (100% - SLO).



2 RELATIONSHIP: SLI, SLO AND SLA



3 EXAMPLE SLI AND SLO

SLI (Availability):

$$\frac{\text{Successful requests}}{\text{Total requests}} \times 100$$

SLO: 99.9% availability (over 30 days)

Error budget:

$$100\% - 99.9\% = 0.1\%$$

If you have 43,200 minutes in 30 days:
0.1% = 43.2 minutes of allowed downtime.

4 ERROR BUDGET VISUALIZATION



When the error budget is consumed, you cannot risk more changes.

5 BURN RATES

- Burn rate shows how fast you are consuming the error budget.
- High burn rate = taking on more risk.
- Common alerting windows:
 - 1 hour (fast burn)
 - 6 hours
 - 24 hours (sustained burn)



Example:

If your monthly error budget is 43 minutes and you have already used 21 minutes in the first day, your burn rate is very high.

6 ALERTING ON SLOs

- Alert on error budget burn rate, not just raw metrics.
- Use multi-window, multi-burn alerts (e.g. 2% in 1 hour and 5% in 6 hours).
- Avoid alerting on every small blip.
- Alerts should drive action, not noise.



Example alert condition (concept):

IF burn_rate > 2% over 1h
AND burn_rate > 5% over 6h
THEN page on-call

7 RELIABILITY VS DELIVERY VELOCITY

High Delivery Velocity	High Reliability
• More changes • Higher risk • Faster feature delivery	• Fewer incidents • Stable performance • Higher customer trust



Goal:

Use the error budget to balance innovation and stability.

8 DEPENDENCY RELIABILITY

- Your SLO can be impacted by external dependencies.
- Understand the reliability of third-party APIs, databases and other services.
- Use timeouts, retries, fallbacks and circuit breakers.
- Have separate SLOs for critical dependencies.



9 ERROR BUDGET POLICIES

- **When error budget is healthy:**
 - Allow normal feature development.
- **When error budget is at risk:**
 - Slow down non-critical changes.
 - Increase testing and review.
 - Focus on reliability improvements.
- **When error budget is exhausted:**
 - Stop non-essential deployments.
 - Only allow critical fixes.
 - Do a full incident review.



10 SENIOR SCENARIO

? Your service is burning its monthly error budget rapidly during the first week. What decisions should follow?

THINGS TO CONSIDER

- ✓ Assess the current burn rate and projected consumption.
- ✓ Identify the cause (recent releases, traffic, dependency issues).
- ✓ Pause or limit non-essential changes.
- ✓ Focus on mitigation and reliability improvements.
- ✓ Communicate with stakeholders.
- ✓ Review SLOs, alerting and release process.

11 INVESTIGATION STEPS

- 1 Check current error budget and burn rate.
- 2 Look at recent changes (deployments, configs).
- 3 Identify which SLI is failing (latency, errors, availability).
- 4 Check internal and external dependencies.
- 5 Correlate with metrics, logs, traces and events.
- 6 Determine blast radius (which users/services are affected).
- 7 Decide on immediate actions (rollback, feature flag, scale).
- 8 Plan long-term fixes and process improvements.



12 KEY TAKEAWAYS

- ✓ Define clear SLIs and SLOs.
- ✓ Monitor and alert on error budget burn rate.
- ✓ Use error budgets to guide release decisions.
- ✓ Balance reliability and delivery velocity.
- ✓ Understand and manage dependency risk.
- ✓ Have clear policies for when the error budget is at risk or exhausted.
- ✓ Continuously review and improve your reliability practices.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

DISTRIBUTED SYSTEMS FAILURE PATTERNS

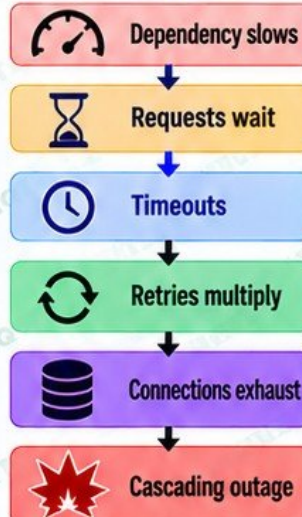
UNDERSTAND HOW FAILURES SPREAD, WHY THEY HAPPEN AND HOW TO PREVENT THEM

1 COMMON FAILURE PATTERNS

- **Retry storms** – retries amplify load.
- **Cascading failure** – one failure spreads.
- **Timeout propagation** – upstream timeouts trigger downstream failures.
- **Thundering herd** – many clients retry at the same time.
- **Queue buildup** – requests pile up in queues.
- **Connection exhaustion** – no available connections.
- **Partial failure** – some components fail, others work.
- **Backpressure** – system cannot accept more load.
- **Circuit breakers** – stop calls to failing services.
- **Exponential backoff and jitter** – retry safely.



2 FAILURE FLOW (HOW IT ESCALATES)



3 REAL-WORLD EXAMPLE



What happens:

- 1 Service B becomes slow (high latency).
- 2 Service A requests wait and timeout.
- 3 Clients retry automatically.
- 4 Retries multiply the load on Service B.
- 5 Connections exhaust.
- 6 Other services start failing.
- 7 Entire system experiences a cascading outage.

4 RETRY STORM

- Clients retry failed requests immediately.
- Retries increase the load on an already struggling service.
- More retries cause longer queues and higher latency.
- Can quickly turn a small issue into a major outage.



5 TIMEOUT PROPAGATION

- An upstream service times out.
- The error is passed to downstream services.
- Each layer adds more timeouts and retries.
- Failure spreads across the system.



6 THUNDERING HERD

- Many clients retry at the same time.
- Happens after a deployment, network blip or service recovery.
- Sudden spike in traffic overwhelms the system.



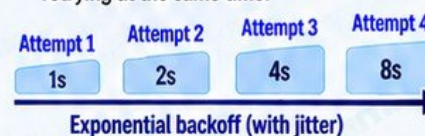
7 CIRCUIT BREAKER

- Monitors failures and stops requests to a failing service.
- Prevents further load and gives the service time to recover.
- After a cooldown period, it allows limited test requests.



8 EXPONENTIAL BACKOFF AND JITTER

- Wait longer between retries.
- Reduces load on the recovering service.
- Jitter (random delay) prevents all clients from retrying at the same time.



9 BACKPRESSURE AND QUEUES

- Queues absorb temporary spikes.
- When queues grow, the system signals clients to slow down.
- Prevents resource exhaustion and keeps the system stable.



10 PARTIAL FAILURE

- Some instances or dependencies fail, others work.
- Can cause inconsistent behavior and hard-to-diagnose issues.
- Requires proper error handling, retries and fallback logic.



11 SENIOR SCENARIO

? Why can retries make a healthy recovery impossible?

THINGS TO CONSIDER

- ✓ How retries increase load during recovery.
- ✓ Impact on queues, connections and resource limits.
- ✓ Interaction with timeouts and circuit breakers.
- ✓ How to design safe retry and backoff strategies.
- ✓ When to fail fast instead of retrying.
- ✓ How to protect downstream dependencies.

12 KEY TAKEAWAYS

- ✓ Understand common distributed failure patterns.
- ✓ Use timeouts, retries and backoff correctly.
- ✓ Implement circuit breakers.
- ✓ Monitor queue depth, latency and error rates.
- ✓ Design for partial failure and graceful degradation.
- ✓ Use backpressure to protect your system.
- ✓ Test failure scenarios regularly.
- ✓ Learn from real incidents and post-mortems.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

SCALING, CAPACITY AND PERFORMANCE ENGINEERING

DESIGN FOR GROWTH, HANDLE PEAK LOADS AND PREVENT PERFORMANCE PROBLEMS

1 HORIZONTAL VS VERTICAL SCALING

Vertical Scaling (Scale Up)



- Increase CPU, memory, Storage on a single instance.
- Simple to implement.
- Limited by instance size.
- Single point of failure risk.

Use cases:

- Small workloads.
- Stateful services (e.g. databases).

Horizontal Scaling (Scale Out)

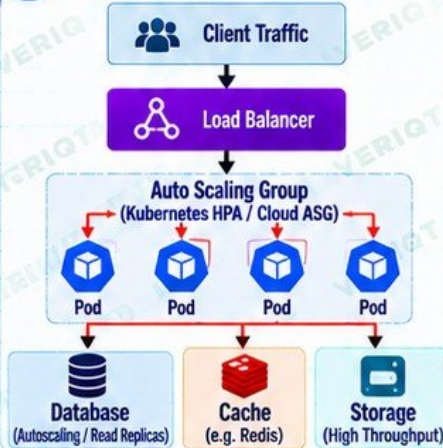


- Add more instances (Pods/servers).
- Improves availability and fault tolerance.
- Handles higher traffic.
- Requires load balancing and stateless design.

Use cases:

- Web applications.
- Microservices.
- Stateless workloads.

2 SCALING ARCHITECTURE EXAMPLE



3 SATURATION AND BOTTLENECKS

- Saturation occurs when a resource reaches its maximum capacity.
- Can happen at CPU, memory, network, storage, database, or external APIs.
- Look for increasing latency, queue length, error rate and rejected requests.
- One saturated component can slow down the entire system.

Common saturation signs:

- High CPU or memory utilization.
- Growing request queue.
- Increased response time (latency).
- Connection limits reached.
- Throttling from external services (API quotas).

4 QUEUEING AND LOAD HANDLING

- Queues absorb traffic spikes.
- Use queueing to smooth out variable load.
- Monitor queue length, processing time and backlog growth.
- Set limits to prevent unbounded queues.



5 AUTOSCALING SIGNALS

- CPU utilization (common but not always enough).
- Memory utilization.
- Request rate (RPS).
- Custom metrics (e.g. queue length, latency).
- External metrics (e.g. API calls, DB connections).
- Scale based on the right signal for your workload.

Example (Kubernetes HPA):

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

6 COLD STARTS

- Cold starts happen when a new instance or Pod takes time to become ready.
- Common in serverless and autoscaling environments.
- Impacts latency during traffic spikes.
- Reduce with:
 - Warm pools / minimum replicas.
 - Optimized images.
 - Faster initialization.
 - Ready checks that reflect real readiness.

Where it happens:

- Serverless functions (e.g. AWS Lambda).
- Autoscaled Pods.
- Managed services with scale-to-zero.

7 DATABASE BOTTLENECKS

- Database can become the limiting factor as you scale.
- Common issues:
 - Connection limits.
 - Slow queries.
 - Lock contention.
 - Disk I/O saturation.
 - Replication lag.



Improve performance:

- Use connection pooling.
- Optimize queries and indexes.
- Use read replicas and caching.
- Monitor DB metrics and slow query logs.

8 STORAGE AND NETWORK THROUGHPUT

- Storage IOPS and throughput can limit application performance.
- Network bandwidth and latency affect cross-service communication.
- Monitor for saturation during peak load.

Storage metrics:

- IOPS
- Throughput (MB/s)
- Read/write latency
- Burst limits (cloud disk types)

Network metrics:

- Bandwidth usage
- Latency
- Packet loss
- Cross-AZ/region latency

9 API QUOTAS AND EXTERNAL LIMITS

- Cloud providers and third-party services enforce API rate limits (quotas).
- Exceeding limits can cause throttling and failures.
- Design with retries, backoff and caching.
- Monitor usage and set alerts.



Examples:

- AWS service limits (API calls, resources).
- GitHub/GitLab API rate limits.
- Payment providers, email services, third-party APIs.

10 METRIC CARDINALITY

- High-cardinality metrics have too many unique label values (e.g. user_id, request_id).
- Can cause high memory usage and slow queries in monitoring systems.
- Use low-cardinality labels and aggregate where possible.

Bad example (high cardinality):

```
http_requests_total(user_id="12345")
```

Better example (low cardinality):

```
http_requests_total(endpoint="/api/order")
```



11 COST GROWTH

- Scaling increases resource usage and cost.
- Watch for uncontrolled autoscaling.
- Optimize resource requests and limits.
- Use cost monitoring and budgets.
- Right-size instances and storage.

Cost control strategies:

- Set budgets and alerts.
- Use reserved/spot instances where appropriate.
- Clean up unused resources.
- Review scaling policies regularly.

12 SENIOR SCENARIO

- CPU-based autoscaling adds Pods, but latency continues increasing. What assumptions might be wrong?

THINGS TO CONSIDER

- ✓ Is CPU the right scaling signal for this workload?
- ✓ Is the database the bottleneck?
- ✓ Are there external dependencies (APIs, services)?
- ✓ Is there a queue backlog?
- ✓ Are you hitting connection limits?
- ✓ Is there a network or storage bottleneck?
- ✓ Are Pods becoming ready but not actually serving traffic?
- ✓ Is there a load balancer or upstream limit?



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

INCIDENT LEADERSHIP AND PRODUCTION RECOVERY

LEAD EFFECTIVELY, RESTORE SERVICE AND TURN INCIDENTS INTO IMPROVEMENTS

1 INCIDENT LIFECYCLE



2 INCIDENT ROLES

- Incident Commander**
Leads the response, makes decisions.
- Communications Lead**
Handles internal and external updates.
- Technical Leads**
Investigate, implement fixes.
- Scribe**
Captures timeline, decisions and actions.
- Stakeholders**
Business, product, security, legal.

3 BLAST RADIUS ANALYSIS

- Which services are affected?
- Which users or regions are impacted?
- Is the issue isolated or widely distributed?
- What downstream services depend on it?
- What is the customer impact?



4 COMMUNICATION

- Be clear, accurate and frequent.
- Share what you know, what you don't know and next update time.
- Tailor messages for technical and non-technical audiences.
- Use a single source of truth (e.g. incident channel).

Example update (internal):

We are investigating increased API errors in the checkout service. Impact started at 14:05 UTC. Next update in 10 minutes.

Example update (external):

We are aware of an issue affecting checkout. Our engineers are actively working on a fix. Next update in 15 minutes.

5 EVIDENCE PRESERVATION

- Capture logs, metrics, traces and events before making changes.
- Record timestamps and who made changes.
- Save relevant configurations and system state.
- Preserve affected data (e.g. query results, error samples).
- Document all investigation steps.

Why it matters:

- Helps with root cause analysis.
- Useful for postmortems and compliance.
- Prevents loss of critical information.

6 MITIGATION VS ROOT CAUSE

Mitigation

- Restores service quickly.
- Reduces or removes customer impact.
- May not fix the underlying cause.
- Focus: get the system stable.

Root Cause

- Finds and fixes the underlying problem.
- May take longer to investigate and implement.
- Prevents recurrence.
- Focus: understand and eliminate the cause.

7 ROLLBACK STRATEGY

- Keep a tested rollback plan.
- Use versioned deployments (e.g. Kubernetes, CI/CD).
- Validate rollback before production use.
- Ensure data compatibility (schema changes).
- Monitor after rollback.

Current Version (v2)

Rollback

Previous Version (v1)

8 ESCALATION

- Define clear escalation criteria.
- Escalate based on impact, not assumptions.
- Involve senior engineers, management or external support when needed.

Examples of escalation triggers:

- Customer-facing outage.
- Widespread impact across services.
- Security or data integrity concerns.
- SLA / error budget at risk.
- Need for external vendor support.

9 RECOVERY VERIFICATION

- Confirm key user journeys work.
- Check metrics (errors, latency, traffic).
- Validate dependent services.
- Monitor for a sustained period.
- Get confirmation from stakeholders.

Verification checklist:

- Functionality working as expected.
- Metrics back to normal range.
- No hidden errors in logs or traces.
- Users confirm issue is resolved.
- Keep monitoring for at least 30-60 minutes.

10 POSTMORTEMS

- Write a blameless postmortem.
- Include timeline, impact, root cause and contributing factors.
- Identify what went well and what didn't.
- Define actionable follow-ups.
- Share learnings across teams.

Postmortem should include:

- Summary and impact.
- Timeline of events.
- Root cause and contributing factors.
- What went well.
- What could be improved.
- Action items with owners and due dates.

11 SENIOR SCENARIO

? You have a probable root cause, but proving it will take 40 minutes. A safe mitigation can restore service in five minutes. What should happen first, and why?

THINGS TO CONSIDER

- Customer impact and priority.
- Risk of further damage.
- Evidence you need to capture before changes.
- How to continue root cause investigation in parallel.
- Communication with stakeholders.
- When to implement the permanent fix.
- How to prevent this from recurring.

SENIOR DEVOPS FINAL ARCHITECTURE INTERVIEW

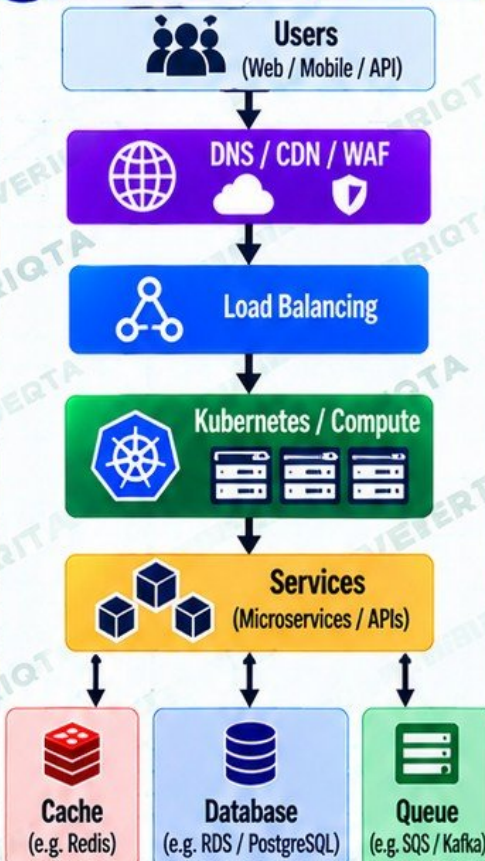
DESIGN A PRODUCTION PLATFORM FOR A BUSINESS-CRITICAL INTERNET-FACING APPLICATION

1 REQUIREMENTS AND USERS

- Understand business goals.
- Define user types and traffic patterns.
- Estimate peak and normal load.
- Identify data sensitivity and compliance.
- Define availability and performance targets.
- List key non-functional requirements (security, cost, scalability, DR).



2 HIGH-LEVEL ARCHITECTURE



3 CI/CD PIPELINE

- Source control and branching strategy.
- Automated testing (build, unit, integration).
- Security scanning (SAST, dependencies, images).
- Artifact registry.
- GitOps or CI/CD (e.g. GitHub Actions, GitLab CI).
- Automated deployments (blue/green, canary).



4 INFRASTRUCTURE AS CODE

- Use Terraform or CloudFormation.
- Version control for infra code.
- Modular and reusable design.
- Separate environments (dev/staging/prod).
- Remote state with locking.
- Plan review and change control.



5 NETWORKING

- VPC / VNet design.
- Public and private subnets.
- NAT for outbound traffic.
- Security groups / firewalls.
- Load balancers and ingress controllers.
- Private connectivity to databases and services.



9 DATABASE, CACHE AND QUEUE

- Choose the right database.
- Read replicas and multi-AZ.
- Caching for performance.
- Queue for async workloads.
- Connection limits and pooling.
- Backup, restore and retention.



10 RELIABILITY AND RESILIENCE

- Multi-AZ / multi-region design.
- Failure domains and blast radius.
- Health checks and self-healing.
- Graceful degradation.
- Fallbacks and circuit breakers.
- Disaster recovery (RPO/RTO).



6 IAM AND SECRETS

- Least privilege access.
- Separate human and workload identity.
- Short-lived credentials.
- Secret storage (e.g. AWS Secrets Manager).
- Rotation and auditability.
- No hardcoded credentials.



7 OBSERVABILITY

- Metrics, logs, traces and events.
- Centralized logging and monitoring.
- SLO and error budget tracking.
- Useful alerts (avoid noise).
- Dashboards for users, services and infra.
- Trace correlation (request IDs).

8 SCALING AND PERFORMANCE

- Horizontal vs vertical scaling.
- Autoscaling (CPU, memory, custom metrics).
- Handle queueing and backpressure.
- Database scaling and read replicas.
- API rate limiting and quotas.



11 DEPLOYMENT AND RECOVERY

- Blue/green or canary deployments.
- Feature flags.
- Rollback strategy.
- Database migration strategy.
- Recovery verification (check service health).
- Runbooks for common failures.



12 COST AND SECURITY

Cost

- Right-size resources.
- Use autoscaling and reserved instances.
- Monitor usage and set budgets.
- Optimize storage, data transfer and logs.

Security

- Defense in depth.
- WAF, DDoS protection.
- Image and dependency scanning.
- Network isolation.
- Encryption in transit and at rest.
- Regular security reviews.



FINAL INTERVIEW SCENARIO



Design a production platform for a business-critical internet-facing application.

THINGS TO EXPLAIN

- ✓ Users and traffic flow.
- ✓ DNS / CDN / WAF.
- ✓ Load balancing.
- ✓ Kubernetes or compute layer.
- ✓ Services architecture.
- ✓ Cache, database and queue.
- ✓ CI/CD pipeline.
- ✓ Infrastructure as code.
- ✓ Networking and security.
- ✓ IAM and secrets.
- ✓ Observability.
- ✓ Scaling and performance.
- ✓ SLOs and error budgets.
- ✓ Failure domains.
- ✓ Multi-AZ resilience.
- ✓ Deployment strategy.
- ✓ Rollback.
- ✓ Backup and disaster recovery.
- ✓ Cost and incident response.

2 CLOSING INTERVIEW QUESTION



What fails first, how would you know, and how would the architecture recover?

WHAT A STRONG ANSWER SHOWS

- ✓ Understands real-world failure modes.
- ✓ Thinks in trade-offs, not just boxes.
- ✓ Designs for resilience and cost efficiency.
- ✓ Explains operational impact.
- ✓ Shows clear monitoring and recovery strategy.
- ✓ Considers security and compliance.
- ✓ Communicates clearly and logically.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta